Guía Teórica: TanStack Query y el Manejo de Server State

1. Introducción: El problema del Frontend Moderno

En las aplicaciones de React tradicionales, el uso de useEffect y fetch para obtener datos genera retos que escalan negativamente:

- **Código repetitivo:** Cada componente debe replicar la lógica de estados de carga, error y datos.
- **Ausencia de caché:** Al desmontar y montar un componente, se dispara una nueva petición innecesaria.
- **Solicitudes duplicadas:** Varios componentes pueden pedir los mismos datos de forma simultánea, saturando la red.
- **Sincronización manual:** Mantener los datos actualizados frente a cambios en el servidor requiere lógica extra muy compleja.

2. ¿Qué es TanStack Query?

Es una librería diseñada específicamente para gestionar el **Server State** (Estado del Servidor). Se encarga de abstraer toda la complejidad del ciclo de vida de los datos remotos.

Diferencia Crítica de Estados

Concepto	Client State	Server State
Definición	Estado interno y local de la UI.	Datos que residen en servidores o APIs externas.
Ejemplos	Modales, formularios, temas (dark/light).	Catálogo de productos, perfiles de usuario.
Desafío	Es fácil de controlar con useState.	Puede quedar desactualizado y requiere sincronización.
Herramienta	useState, Zustand, Redux.	**TanStack Query**, SWR.

3. Configuración del Entorno

Para implementar TanStack Query, se siguen tres pasos fundamentales:

1. **Instalación:** pnpm add @tanstack/react-query.
2. **Creación del Cliente:** Se instancia un QueryClient para gestionar la caché global.
3. **Provider:** Se envuelve la aplicación con QueryClientProvider para dar acceso al contexto en todo el árbol de componentes.

4. Implementación Técnica (Código)

A. Definición de Tipos y Acciones (API)

Es fundamental tipar los datos para obtener autocompletado y detectar errores en tiempo de compilación.

TypeScript

```
// types/product.ts

export interface Product {

  id: number;

  title: string;

  price: number;

  description: string;

  category: string;

  image: string;

}


// api/products.actions.ts

export const getProducts = async (): Promise<Product[]> => {

  const res = await fetch("https://fakestoreapi.com/products");


  // Es vital lanzar un error explícito para que useQuery detecte isError [cite: 111, 112]

  if (!res.ok) {

    throw new Error("Error al cargar productos"); [cite: 109]

  }
```

```
  return res.json(); [cite: 118]
};
```

B. Uso del Hook useQuery

El hook useQuery es la herramienta principal para leer datos. Requiere una queryKey (identificador único para la caché) y una queryFn (la función que hace la petición).

TypeScript

```
import { useQuery } from "@tanstack/react-query";
import { getProducts } from "../api/products.actions";
import type { Product } from "../types/product";

export const ProductsPage = () => {
  // El hook devuelve estados automáticos: data, isLoading, isError [cite: 80, 81]
  const { data, isLoading, isError } = useQuery<Product[]>({
    queryKey: ["products"], [cite: 82]
    queryFn: getProducts, [cite: 83]
  });

  if (isLoading) return <p>Cargando...</p>; [cite: 133]
  if (isError) return <p>Error al cargar datos</p>; [cite: 134]

  return (
    <div className="p-8">
      <h1 className="text-3xl font-bold">Productos</h1>
      <div className="grid grid-cols-3 gap-6">
        {data?.map((product) => (
          <div key={product.id} className="shadow-md p-5 rounded-2xl">
```

```
            <h2 className="font-semibold">{product.title}</h2>
          </div>
        ))}
      </div>
    </div>
  );
};
```

5. Beneficios y Funcionamiento Automático

TanStack Query transforma el flujo de trabajo de la siguiente manera:

- **Verificación de caché:** Antes de hacer un fetch, revisa si ya existen datos para esa queryKey.
- **Deduplicación:** Si dos componentes piden los mismos datos, solo se realiza una petición al servidor.
- **Refetch en segundo plano:** Actualiza los datos automáticamente cuando el usuario vuelve a la pestaña o recupera la conexión.
- **Menos Boilerplate:** Elimina la necesidad de múltiples estados manuales, haciendo el código más legible y mantenible.